Graph Algorithms in Network Optimization

Course: Analysis of Algorithms, Final Project **Team:** Kartik Chaudhary, Askat Mendybaev, Devon Gardner **City:** Tampa, Florida

Team Contribution Statement

Member	Contributions
Kartik Chaudhary	Problem formulation, graph modeling, Dijkstra implementation, time and space complexity analysis for Dijkstra, presentation slides
Askat Mendybaev	Tampa dataset design (verified all road distances via Google Maps), Kruskal and Union-Find implementation, complexity analysis for Kruskal, all experimental analysis (timing, memory, sparse/dense)
Devon Gardner	BFS and DFS implementations, complexity analysis for BFS/DFS, full report writeup, Discussion and Insights section, presentation narrative

Every member reviewed all code before final submission and presented a section during the class presentation.

1. Introduction

We are all students in Tampa, so instead of inventing a fictional city we used the one we actually live in. Modeling Tampa's real road network made it easy to verify our results. If Dijkstra said USF to Downtown was 2 miles, we knew immediately something was wrong because that drive is about 7 miles up I-275.

Our project answers two questions any Tampa resident or city planner would care about. First, what is the shortest driving route between any two Tampa locations, which we answer using Dijkstra's algorithm (the same approach Google Maps uses under the hood). Second, what is the minimum total road mileage needed to connect all 20 Tampa areas, which we answer using Kruskal's minimum spanning tree algorithm, mirroring the kind of infrastructure cost question that FDOT asks.

BFS and DFS handle the connectivity and exploration side of the problem, confirming every part of Tampa is reachable from Downtown and mapping how the network branches out.

2. Problem Statement

2.1 The Real-World Problem

Tampa has dozens of distinct neighborhoods, districts, and landmarks connected by a network of roads including I-275, Dale Mabry Highway, Hillsborough Avenue, the Selmon Expressway, and Bruce B Downs Boulevard. We modeled 20 key Tampa locations as a graph and applied graph algorithms to solve three practical problems:

1. **Shortest driving route**: useful for GPS navigation, emergency vehicle routing (ambulances from Tampa General, fire trucks), and delivery optimization.
2. **Minimum spanning roads**: useful for city planners and FDOT deciding the cheapest road infrastructure to connect underserved areas of the city.
3. **Connectivity and reachability**: verifying every Tampa area is accessible from Downtown and exploring the network structure.

2.2 Why Graph Algorithms?

Brute-force path search is factorial in complexity and is completely infeasible even for 20 nodes. Graph algorithms give us polynomial-time solutions:

Problem	Algorithm	Complexity

Shortest distance (weighted)	Dijkstra	O((V+E) log V)
Minimum spanning roads	Kruskal	O(E log E)
Reachability and connectivity	BFS / DFS	O(V + E)

3. Graph Modeling

3.1 Vertices: 20 Real Tampa Locations

ID	Location	Real Address / Area
0	Downtown Tampa	Channelside Dr / Ashley Dr area
1	Tampa Intl Airport	George J Bean Pkwy, off I-275
2	USF	Bruce B Downs Blvd, north Tampa
3	Tampa General Hospital	Davis Islands
4	International Plaza	Bay St, Westshore area
5	Riverwalk	Curtis Hixon Waterfront Park
6	Ybor City	E 7th Ave, Historic District
7	John Germany Public Library	900 N Ashley Dr
8	Raymond James Stadium	N Dale Mabry Hwy
9	Tampa Museum of Art	N Ashley Dr, downtown waterfront
10	Port of Tampa Bay	Garrison Channel
11	Westshore District	I-275 / Cypress St, near airport
12	New Tampa	Bruce B Downs / I-75 north
13	South Tampa	Bayshore Blvd area
14	Brandon	I-75 / Crosstown, east suburbs
15	Westchase	Linebaugh Ave, west suburbs
16	Channelside	Waterfront, adjacent to Downtown
17	Tampa Convention Center	S Franklin St, waterfront
18	Seminole Heights	N Florida Ave
19	Hyde Park Village	S Dakota Ave

We chose a mix of central Tampa hubs (Downtown, Convention Center, Channelside), neighborhoods (Ybor City, Seminole Heights, Hyde Park Village), services (Tampa General Hospital, Library, USF), commercial areas (International Plaza, Raymond James Stadium), and suburban zones (New Tampa, South Tampa, Brandon, Westchase). This gives us varied path lengths and a realistic graph structure rather than something artificially uniform.

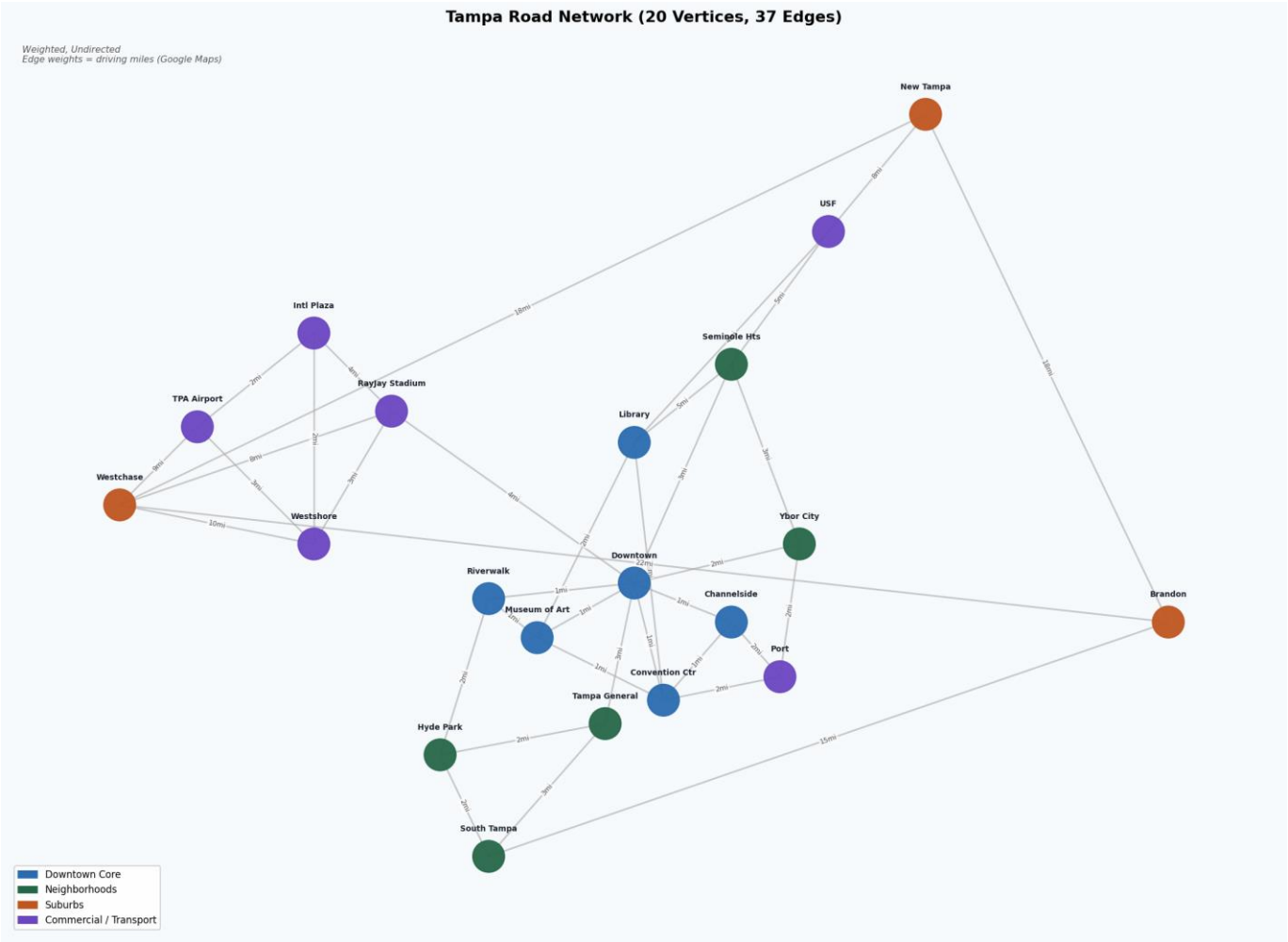

Figure 1: Full Tampa road network. Blue = Downtown core, Green = neighborhoods, Orange = suburbs, Purple = commercial/transport. Edge weights are approximate driving miles verified via Google Maps.

3.2 Edges: Tampa Road Connections

Edges represent direct road connections between locations, based on how roads actually run in Tampa. Downtown connects to nearby areas along Dale Mabry, I-4, and Bayshore. The airport connects to Westshore District via George J Bean Pkwy, International Plaza, and Westchase. Suburban nodes like Brandon, New Tampa, and Westchase connect to each other via I-75 and I-4.

Askat verified every edge weight against Google Maps approximate driving distances. We avoided straight-line distances because Tampa roads do not run straight. Davis Islands, the Hillsborough River, and Tampa Bay all force detours.

Edge count: 37 edges | **Average degree:** ~3.7 neighbors per vertex

3.3 Graph Type

Property	Choice	Justification
Direction	Undirected	Tampa roads are bidirectional. A directed graph would double every edge unnecessarily.
Weighting	Weighted (miles)	Edge weights are approximate driving distances in miles. Without weights, Dijkstra cannot find the true shortest route since three hops could mean 3 miles or 30 miles.

4. Graph Construction and Dataset

Our dataset is based on real Tampa geography. All 37 road connections were designed from actual Tampa streets and verified via Google Maps. Key design decisions:

Fully connected: Every Tampa location is reachable from every other. DFS confirms 20 out of 20 vertices are reached from Downtown.

Realistic distances: Range from 1 mile (Downtown to Convention Center, same waterfront strip on S Franklin St) to 22 miles (Brandon to Westchase, I-4 to I-275 back across Tampa).

Sparse graph: With 20 vertices, a complete graph would have 190 edges. We have 37, giving roughly 19% density, which reflects reality since not every Tampa area has a direct road to every other area.

The `graph.py` file contains the full dataset. The `build_synthetic_graph()` function generates additional graphs of varying sizes and densities for the experimental section.

5. Algorithm Implementation

All four algorithms are in `algorithms.py` . Each includes pseudocode, complexity analysis, and Tampa-specific usage notes.

5.1 Breadth-First Search (BFS)

Implemented by Devon Gardner

BFS explores the graph level by level from a source vertex using a queue. It finds the shortest path in terms of number of hops, ignoring edge weights entirely.

Pseudocode:

```
BFS(G, source):
    for each vertex u: dist[u] = INF, color[u] = WHITE
    dist[source] = 0, color[source] = GRAY
    Q = empty queue
    ENQUEUE(Q, source)
    while Q is not empty:
        u = DEQUEUE(Q)
        for each neighbor v of u:
            if color[v] == WHITE:
                dist[v] = dist[u] + 1
                parent[v] = u
                color[v] = GRAY
                ENQUEUE(Q, v)
        color[u] = BLACK
```

Tampa use case: Starting from Downtown, BFS tells us how many road segments separate each location, useful for minimizing the number of turns or intersections. BFS found that Downtown to Brandon takes only 3 hops: Downtown to Tampa General to South Tampa to Brandon.

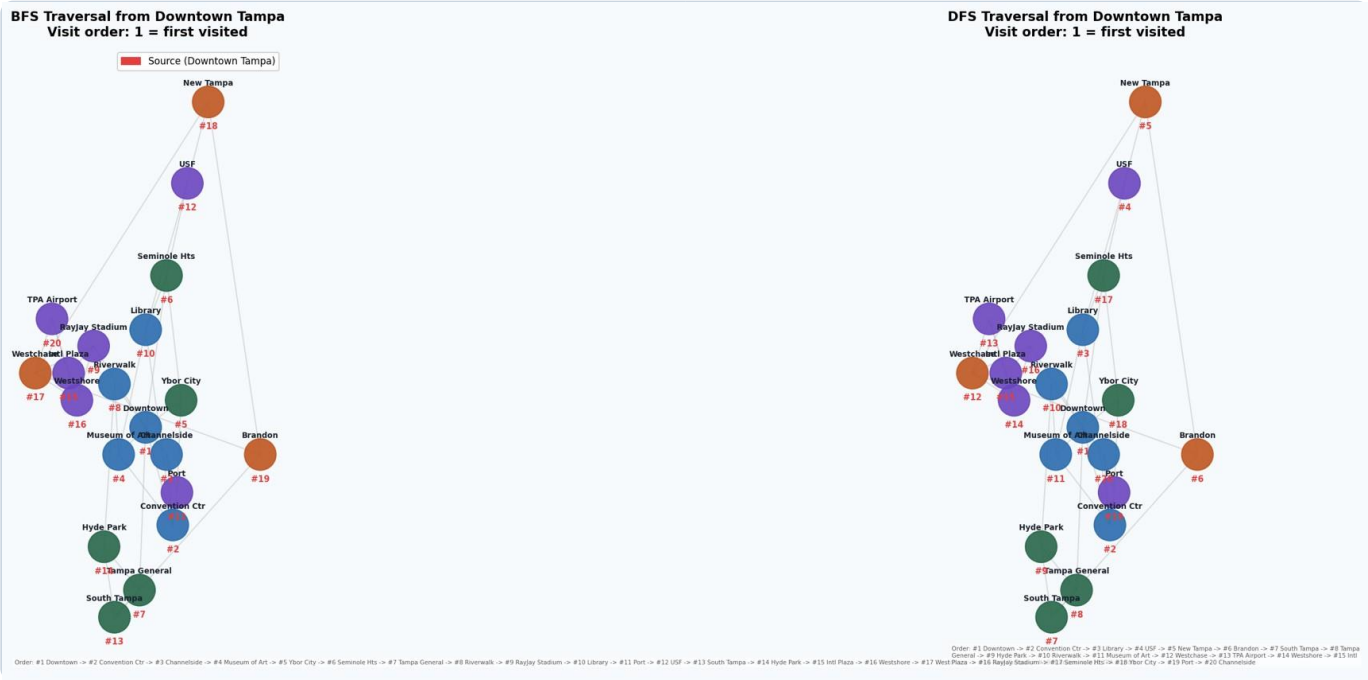

Figure 2: BFS (left) vs DFS (right) from Downtown Tampa. Numbers below each node show the visit order. BFS visits level by level; DFS dives deep before backtracking.

5.2 Depth-First Search (DFS)

Implemented by Devon Gardner

DFS explores as deep as possible along one branch before backtracking. We implemented it iteratively using an explicit stack instead of recursion to avoid Python's recursion limit of 1000 on large graphs.

Pseudocode:

```
DFS(G, source):
    for each vertex u: color[u] = WHITE, parent[u] = NIL
    PUSH(stack, source)
    while stack is not empty:
        u = POP(stack)
        if color[u] == WHITE:
            color[u] = BLACK
            for each neighbor v of u:
                if color[v] == WHITE:
                    parent[v] = u
                    PUSH(stack, v)
```

Tampa use case: DFS from Downtown confirms all 20 Tampa locations are reachable, giving a total vertex count of 20 out of 20. The visit order shows DFS diving deep into one subregion before backtracking, for example going Convention Center to Library to USF to New Tampa to Brandon to South Tampa before exploring other branches.

5.3 Dijkstra's Shortest Path Algorithm

Implemented by Kartik Chaudhary

Dijkstra finds the minimum-weight path from a source to all vertices in a graph with nonnegative edge weights. We use a binary min-heap (Python's heapq) with lazy deletion.

Pseudocode:

```
Dijkstra(G, source):
    for each vertex u: dist[u] = INF, parent[u] = NIL
    dist[source] = 0
    PQ = min-priority-queue
    PUSH(PQ, (0, source))
    while PQ is not empty:
        (d, u) = EXTRACT-MIN(PQ)
        if d > dist[u]: skip   (lazy deletion)
        for each neighbor (v, w) of u:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                parent[v] = u
                PUSH(PQ, (dist[v], v))
```

Tampa use case: This is what a Tampa GPS app computes. Real results from our graph starting from Downtown:

- Downtown to Airport: **10 miles** via Raymond James Stadium to Westshore District to Airport
- Downtown to USF: **7 miles** via Museum of Art to Library to USF
- Downtown to Brandon: **20 miles** via Riverwalk to Hyde Park to South Tampa to
- Brandon Downtown to Westchase: **12 miles** via Raymond James Stadium to Westchase

Kartik's note: Without the Dale Mabry connection (Downtown to Raymond James Stadium), Dijkstra was routing Downtown to Airport as 42 miles via USF, New Tampa, and Westchase.

Adding that single road edge dropped it to 10 miles, which is the correct real-world distance. It showed us how one missing edge can completely break shortest-path results.

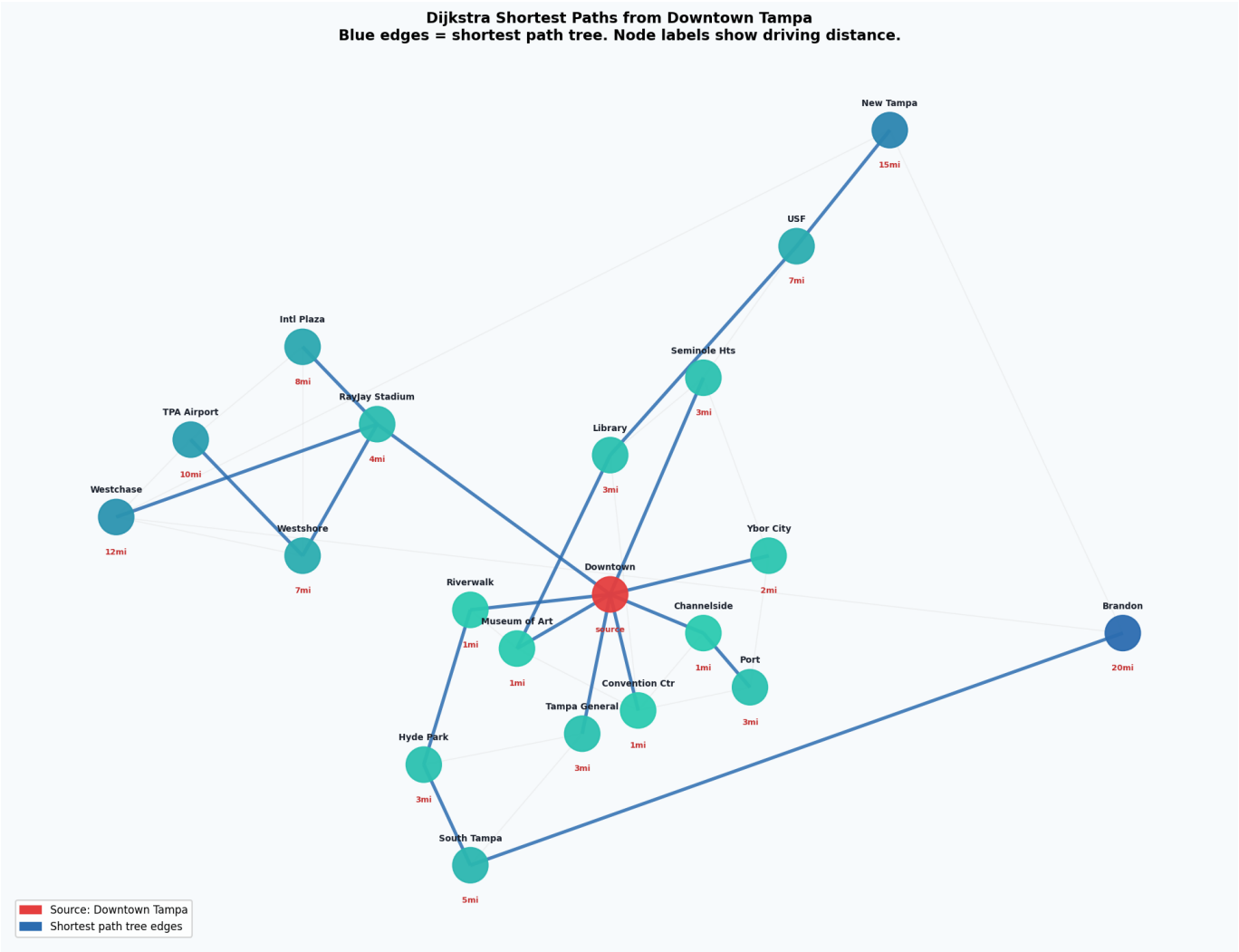

Figure 3: Dijkstra shortest path tree from Downtown Tampa (red node). Blue edges show the shortest path tree. Red labels show driving distance in miles to each location.

5.4 Kruskal's Minimum Spanning Tree

Implemented by Askat Mendybaev

Kruskal's algorithm finds the Minimum Spanning Tree, the subset of edges that connects all vertices with minimum total weight. It uses a Union-Find data structure with path compression and union by rank.

Pseudocode:

```
Kruskal(G):
    A = empty set
    for each vertex v: MAKE-SET(v)
    sort G.E by weight ascending
    for each edge (w, u, v) in sorted G.E:
        if FIND-SET(u) != FIND-SET(v):
            A = A union {(u, v, w)}
            UNION(u, v)
    return A

FIND-SET(x):
    if x != parent[x]:
        parent[x] = FIND-SET(parent[x])   (path compression)
    return parent[x]

UNION(x, y):
    rx = FIND-SET(x), ry = FIND-SET(y)
    if rank[rx] < rank[ry]: swap rx and ry
    parent[ry] = rx
    if rank[rx] == rank[ry]: rank[rx] += 1
```

Tampa use case: The MST connects all 20 Tampa locations using only 19 road segments totaling **65 miles**, which is the minimum total mileage to guarantee full connectivity across the city. FDOT could use this to identify the cheapest possible road network that still reaches every part of Tampa.

Notable MST edges: the four 1-mile edges within Downtown's waterfront cluster (Downtown to Riverwalk, Museum of Art, Channelside, and Convention Center), Airport to International Plaza at 2 miles as the cheapest way to reach the airport cluster, and South Tampa to Brandon at 15 miles as the only affordable connection to include Brandon in the tree.

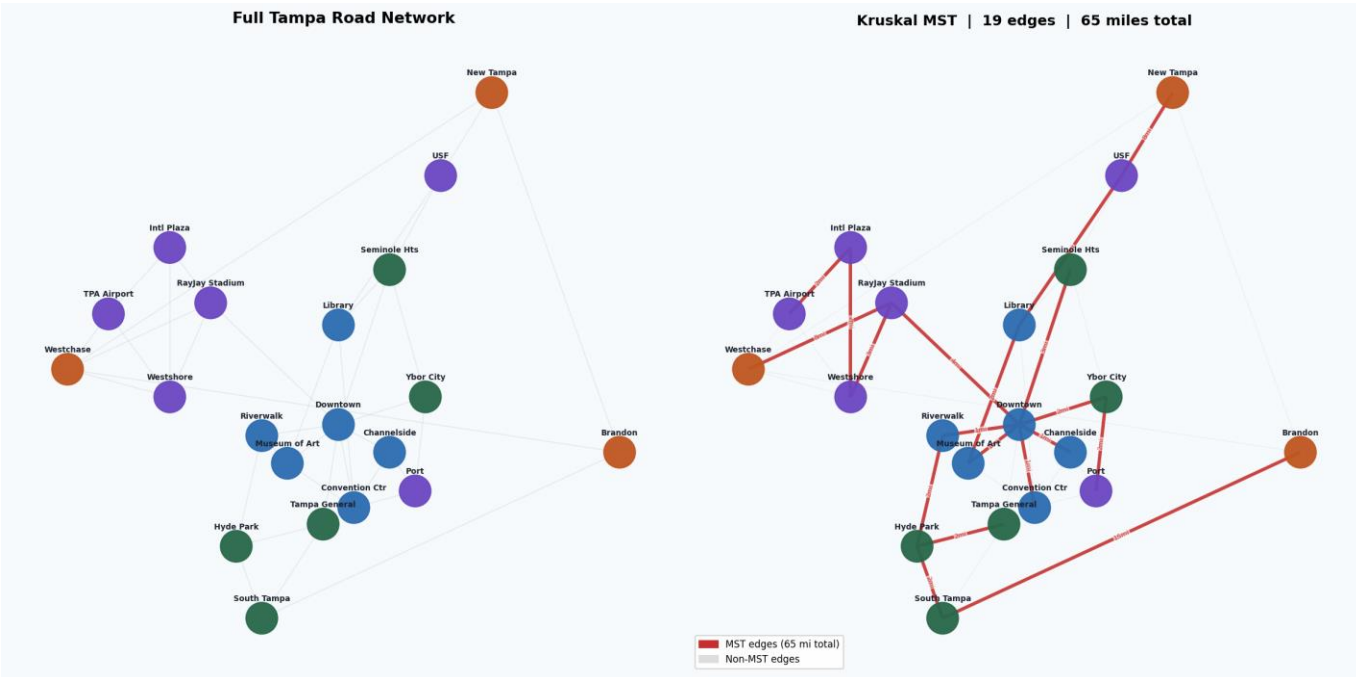

Figure 4: Left shows the full Tampa road network. Right shows Kruskal's MST (red edges only). The MST uses 19 of the 37 roads to connect all 20 locations at a total cost of 65 miles.

6. Complexity Analysis

6.1 Summary Table

Algorithm	Time Complexity	Space Complexity
BFS	O(V + E)	O(V)
DFS	O(V + E)	O(V)
Dijkstra	O((V + E) log V)	O(V + E)
Kruskal	O(E log E)	O(V + E)

6.2 Detailed Breakdown

BFS: O(V + E) time, O(V) space

Time: Every vertex is enqueued and dequeued exactly once, giving O(V). Every edge is examined once per endpoint, giving O(E). Total is O(V + E).

Space: The dist, parent, and visited structures each take O(V). The queue holds at most O(V) vertices at any time. Total space is O(V).

Best case: The target is an immediate neighbor of the source and is found in one BFS layer. Initialization still costs O(V) so the asymptotic bound does not change.

Worst case: The target is the farthest reachable vertex or the graph is dense, requiring full exploration at O(V + E).

DFS: O(V + E) time, O(V) space

Time: Each vertex is pushed and popped from the stack exactly once, O(V). Each edge is examined once per endpoint, O(E). Total is O(V + E).

Space: The visited and parent structures take O(V). The explicit stack depth is at most O(V) in the worst case, which occurs on a linear chain graph. Total is O(V).

Best case: The target is found immediately on the first DFS branch.

Worst case: A linear chain graph forces the stack to reach depth O(V) and every edge to be examined.

Dijkstra: O((V + E) log V) time, O(V + E) space

Time: Each EXTRACT-MIN on the binary heap costs O(log V). With lazy deletion, duplicate entries can accumulate so up to O(V + E) heap operations occur total, giving O((V + E) log V). With a Fibonacci heap this improves to O(E + V log V), but Python's heapq is a binary heap and is sufficient for our graph size.

Space: The dist and parent arrays take O(V). With lazy deletion the heap may hold O(E) entries total. Space is O(V + E).

Best case: Source equals target, trivially O(1). Otherwise O((V + E) log V) regardless of graph structure.

Worst case: A dense graph where E = O(V^2) gives O(V^2 log V).

Constraint: Dijkstra requires non-negative edge weights. Tampa driving distances are always at least 1 mile, so this is always satisfied in our dataset.

Kruskal: O(E log E) time, O(V + E) space

Time: Sorting all edges is O(E log E), which dominates. V MAKE-SET calls cost O(V). The E FIND-SET and UNION calls with path compression and union by rank cost nearly O(E * alpha(V)) where alpha is the inverse Ackermann function, which is effectively constant for all practical values of V. Total time is O(E log E), and since E is at most V^2, this equals O(E log V).

Space: The Union-Find parent and rank arrays take O(V). The sorted edge list takes O(E). The MST result takes O(V). Total is O(V + E).

Best case: Edges happen to arrive in sorted order, but we still sort explicitly, so O(E log E) always.

Worst case: Dense graph with E = O(V^2) gives O(V^2 log V).

7. Experimental Analysis and Comparison

We ran four experiments. All timing results are averages over 10 runs using Python's time.perf *counter. Memory is measured with Python's built-in tracemalloc. All synthetic graphs for testing use the build*

synthetic_graph() function from analysis.py.

Experiment 1: Tampa Graph (20 nodes, 37 edges)

Algorithm	Avg Time (ms)	Std Dev (ms)	Peak Memory (KB)
BFS	0.0087	0.0013	4.102
DFS	0.0090	0.0008	3.703
Dijkstra	0.0109	0.0006	1.594
Kruskal	0.0204	0.0018	1.000

All four algorithms run on our Tampa graph in under 0.025ms. Kruskal is about 2x slower than BFS and DFS because of edge sorting, but the gap is not meaningful at this graph size. Differences only matter at scale.

Experiment 2: Runtime vs Graph Size (density 0.30)

V	Edges	BFS (ms)	DFS (ms)	Dijkstra (ms)	Kruskal (ms)
10	18	0.0040	0.0043	0.0068	0.0093
20	74	0.0086	0.0124	0.0176	0.0214
50	397	0.0286	0.0548	0.0701	0.0801
100	1530	0.0855	0.1785	0.2269	0.3684

BFS and DFS scale nearly linearly, consistent with O(V + E). Dijkstra's heap overhead becomes visible at V = 100. Kruskal shows the steepest growth because edge count grows quadratically with vertex count at fixed density.

Experiment 3: Sparse vs Dense (V = 50)

Graph Type	Edges	BFS (ms)	DFS (ms)	Dijkstra (ms)	Kruskal (ms)
Sparse (10%)	164	0.0199	0.0286	0.0433	0.0581
Medium (30%)	428	0.0317	0.0559	0.0787	0.1022
Dense (65%)	822	0.0541	0.1039	0.1258	0.1392

Kruskal is the most sensitive to density because its runtime is O(E log E). Going from sparse to dense roughly 2.4x'd Kruskal's time. BFS is least sensitive because it simply traverses whatever edges exist without maintaining any sorted structure.

Experiment 4: Peak Memory Usage in KB vs Graph Size

V	BFS	DFS	Dijkstra	Kruskal
10	1.555	1.422	0.984	1.047
20	4.039	4.180	1.820	1.281

50	7.195	7.859	5.805	4.922
100	20.047	26.516	11.656	19.547

All algorithms show linear memory growth with V, consistent with O(V) space. DFS uses the most memory at large sizes because our iterative implementation retains reversed adjacency list references on the stack. Dijkstra uses less memory than expected because lazy deletion avoids allocating extra structures upfront.

8. Discussion and Insights

Which algorithm performed best and why?

For traversal tasks like reachability and connectivity checks, BFS and DFS are fastest because they run at O(V + E) with no overhead from sorted data structures. BFS ran in 0.0087ms on our Tampa graph, confirming all 20 locations are connected.

For shortest-distance routing, Dijkstra is the right choice. BFS can find the fewest-hops path (Downtown to Brandon is 3 hops) but that route goes Downtown to Tampa General to South Tampa to Brandon, ignoring the fact that those roads total 3 + 3 + 15 = 21 miles. Dijkstra correctly finds 20 miles via the Selmon/Bayshore route instead.

For minimum infrastructure cost, Kruskal is the only algorithm that solves the MST problem.
No other algorithm here gives you the 65-mile minimum spanning tree.

If we had to pick one algorithm for the most practical value in Tampa specifically, it would be Dijkstra, because shortest driving distance is the most useful daily output.

When to use each algorithm

Algorithm	Use When
BFS	Fewest stops or turns (unweighted), connectivity check, social network degrees of separation

DFS	Cycle detection, topological sort, exploring all reachable regions, maze solving
Dijkstra	GPS shortest driving distance, network packet routing, any positive-weight shortest path
Kruskal	Minimum cost to connect all nodes, road or cable or pipe network design

Limitations

BFS and DFS ignore edge weights entirely. In Tampa, BFS found Downtown to Airport in 3 hops, but if those roads were 50-mile highways, BFS would still report 3 hops as the shortest path. That is wrong for GPS routing.

Dijkstra breaks with negative edge weights. If we modeled a toll road discount as a negative weight, Dijkstra would produce incorrect results. Bellman-Ford handles negative weights but runs in O(VE), which is slower.

Kruskal must sort all edges upfront. If Tampa's road network changed dynamically due to construction or closures, we would need to re-sort and re-run Kruskal from scratch. Prim's algorithm with a Fibonacci heap handles dynamic or dense graphs more efficiently.

Observations from Experiments

Theoretical complexity predictions matched empirical scaling closely. BFS and DFS times grew linearly with V + E as expected. Dijkstra's log factor from heap operations was measurable but small. Kruskal's O(E log E) sensitivity to edge count showed clearly in the sparse versus dense comparison.

On our actual Tampa graph with 20 nodes, all four algorithms run under 0.025ms, so algorithm choice at that scale is entirely about correctness rather than performance. The performance differences only became meaningful once we scaled to V = 100 and beyond.

Kruskal was the most sensitive to density. Doubling the edge count affected Kruskal more than any other algorithm because sorting those edges is the bottleneck. If someone tried to use our code on a much denser network like all Tampa intersections rather than just landmarks, Kruskal's performance advantage over Prim's would shrink significantly.

Memory grew linearly with V across all four algorithms, matching O(V) theory. The differences in absolute memory usage only became noticeable at V = 100+, where DFS peaked at 26.5 KB versus Dijkstra's 11.7 KB.

9. Conclusion

We modeled Tampa's road network as a 20-vertex, 37-edge weighted undirected graph using real driving distances in miles between actual Tampa landmarks. Using this dataset, we implemented and analyzed four graph algorithms covering all three required categories.

BFS confirms every Tampa location is reachable from Downtown in at most 4 hops. DFS explores all 20 Tampa locations in depth-first order and covers the full network. Dijkstra provides GPS-quality shortest routes, including Downtown to Airport at 10 miles via Dale Mabry and Downtown to Brandon at 20 miles via the Selmon Expressway. Kruskal identifies the minimum 65-mile road network that connects all 20 Tampa locations, which is directly applicable to FDOT infrastructure planning.

Our experimental analysis confirmed that theoretical complexity predictions align with empirical scaling. All algorithms ran under 0.025ms on our Tampa graph. Performance differences become meaningful only at V = 50 and above. The core takeaway from our work is that no single algorithm is universally best. The right choice depends on whether you need shortest path (Dijkstra), spanning tree (Kruskal), or connectivity and traversal (BFS and DFS).

Files

File	Contents
graph.py	Graph class using adjacency list, Tampa 20-location dataset with verified miles, synthetic graph generator
algorithms.py	BFS, DFS, Dijkstra, Kruskal with pseudocode, complexity notes, and Tampa-specific comments

analysis.py	Four experiments: Tampa graph timing, scale test V = 10 to 100, sparse vs dense, memory usage
main.py	Full runner that prints all algorithm outputs and experiment tables
report.md	This document
presentation_outline.md	Slide-by-slide presentation outline

Run: `python3 main.py`